

Praktikum Mandiri 07 | ML Senin Pagi

Nama : Rika Rahma

NIM : 011022214

Rombel : TI08 - 2022

1. Import Library

```
# Library digunakan untuk analisis data, visualisasi, preprocessing, dan evaluasi model.
import pandas as pd          # untuk manipulasi data
import numpy as np          # untuk perhitungan numerik
import matplotlib.pyplot as plt # untuk visualisasi hasil
import seaborn as sns       # untuk visualisasi tambahan (opsional)
from sklearn.model_selection import train_test_split # untuk membagi data latih & uji
from sklearn.preprocessing import StandardScaler    # untuk standarisasi fitur numerik
from sklearn.metrics import r2_score, mean_squared_error # untuk evaluasi model
import statsmodels.api as sm      # untuk membangun model OLS
from statsmodels.stats.outliers_influence import variance_inflation_factor # untuk cek multikolinearitas
```

2. Load Dataset

```
# Menghubungkan colab dengan Google Drive
from google.colab import drive
drive.mount('/content/gdrive')
```

Mounted at /content/gdrive

```
# Memanggil dataset lewat Gdrive
path = '/content/gdrive/MyDrive/Praktikum_ML/praktikum07/data/'
```

```
# Membaca file CSV menggunakan pandas
df = pd.read_csv(path + 'dataset_satelit.csv')
df.head()
```

	No	Longitude	Latitude	N	P	K	Ca	Mg	Fe	Mn	...	b1	Sigma_VV	Sigma_VH	plia	lia	i
0	1	103.036658	-0.604417	2.64	0.15	0.415	0.51	0.31	119.96	463.23	...	0.0433	0.18183	0.04461	35.74446	35.79744	35.41
1	2	103.037201	-0.604689	2.75	0.17	0.568	0.76	0.58	102.63	493.81	...	0.0465	0.22079	0.04640	35.12096	35.14591	35.41
2	3	103.036359	-0.603012	1.77	0.12	0.339	0.49	0.6	107.37	460.93	...	0.0417	0.18926	0.03992	35.07724	35.07730	35.41
3	4	103.036950	-0.603219	2.30	0.15	0.460	0.74	0.67	96.02	338.17	...	0.0367	0.14769	0.03622	36.08078	36.08469	35.41
4	5	103.036802	-0.601969	2.05	0.14	0.308	0.64	0.72	87.01	384.33	...	0.0361	0.18205	0.03797	32.68855	32.69293	35.41

5 rows × 34 columns

```
# Cek informasi kolom dan tipe data
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 594 entries, 0 to 593
Data columns (total 34 columns):
#   Column      Non-Null Count  Dtype
---  -
0   No           594 non-null    int64
1   Longitude    594 non-null    float64
2   Latitude     594 non-null    float64
3   N            594 non-null    float64
4   P            594 non-null    float64
5   K            593 non-null    float64
6   Ca           594 non-null    float64
7   Mg           594 non-null    object
8   Fe           594 non-null    float64
9   Mn           594 non-null    float64
```

```

10 Cu          594 non-null float64
11 Zn          594 non-null float64
12 B           594 non-null float64
13 b12         594 non-null float64
14 b11         594 non-null float64
15 b9          594 non-null float64
16 b8a         594 non-null float64
17 b8          594 non-null float64
18 b7          594 non-null float64
19 b6          594 non-null float64
20 b5          594 non-null float64
21 b4          594 non-null float64
22 b3          594 non-null float64
23 b2          594 non-null float64
24 b1          594 non-null float64
25 Sigma_VV    594 non-null float64
26 Sigma_VH    594 non-null float64
27 plia        594 non-null float64
28 lia         594 non-null float64
29 iafe        594 non-null float64
30 gamma0_vv   594 non-null float64
31 gamma0_vh   594 non-null float64
32 beta0_vv    594 non-null float64
33 beta0_vh    594 non-null float64
dtypes: float64(32), int64(1), object(1)
memory usage: 157.9+ KB

```

```
df.describe()
```

	No	Longitude	Latitude	N	P	K	Ca	Fe	Mn	Cu	...
count	594.000000	594.000000	594.000000	594.000000	594.000000	593.000000	594.000000	594.000000	594.000000	594.000000	...
mean	297.500000	106.878644	-1.024933	2.259091	0.141380	0.582175	0.595094	74.613771	308.034697	2.391195	...
std	171.617307	4.949840	0.965349	0.395499	0.019782	0.222567	0.366118	55.579655	241.731643	1.580296	...
min	1.000000	102.760857	-2.333750	1.140000	0.090000	0.122000	0.050000	21.080000	3.160000	0.090000	...
25%	149.250000	102.927811	-2.233338	1.982500	0.130000	0.429000	0.320000	40.705000	124.015000	1.172500	...
50%	297.500000	103.581969	-0.602276	2.280000	0.140000	0.549000	0.540000	65.650000	239.445000	2.225000	...
75%	445.750000	113.403797	-0.257349	2.570000	0.150000	0.710000	0.790000	87.372500	434.990000	3.357500	...
max	594.000000	113.434700	0.069251	3.230000	0.220000	1.489000	2.820000	559.100000	2009.320000	8.170000	...

8 rows × 33 columns

```
print(df.columns)
```

```

Index(['No', 'Longitude', 'Latitude', 'N', 'P', 'K', 'Ca', 'Mg', 'Fe', 'Mn',
      'Cu', 'Zn', 'B', 'b12', 'b11', 'b9', 'b8a', 'b8', 'b7', 'b6', 'b5',
      'b4', 'b3', 'b2', 'b1', 'Sigma_VV', 'Sigma_VH', 'plia', 'lia', 'iafe',
      'gamma0_vv', 'gamma0_vh', 'beta0_vv', 'beta0_vh', 'NDVI', 'SWIR_ratio',
      'K_proxy'],
      dtype='object')

```

3. Tentukan Target dan Fitur

```

# Fitur: gabungan data geospasial, optik (b1-b12), radar (Sigma, Gamma, Beta),
# serta atribut tambahan (plia, lia, iafe)
target = 'K'
feature_cols = [
    'Longitude', 'Latitude',
    'b1', 'b2', 'b3', 'b4', 'b5', 'b6', 'b7', 'b8', 'b8a', 'b9', 'b11', 'b12',
    'Sigma_VV', 'Sigma_VH', 'gamma0_vv', 'gamma0_vh', 'beta0_vv', 'beta0_vh',
    'plia', 'lia', 'iafe'
]

print(f"Target (y): 'K', Fitur (X): {len(feature_cols)} kolom Radar.")

```

```
Target (y): 'K', Fitur (X): 23 kolom Radar.
```

4. Feature Engineering

```
# Membuat fitur turunan yang memiliki hubungan potensial dengan unsur Kalium.
# NDVI: indeks vegetasi dari band NIR (b8) dan merah (b4)
# SWIR_ratio: rasio band SWIR (b11 / b12) untuk mendeteksi kelembapan
# K_proxy: interaksi antara SWIR dan radar VV sebagai proksi Kalium
df['NDVI'] = (df['b8'] - df['b4']) / (df['b8'] + df['b4'] + 1e-8)
df['SWIR_ratio'] = df['b11'] / (df['b12'] + 1e-8)
df['K_proxy'] = df['b11'] * df['Sigma_VV']

# Tambahkan fitur hasil rekayasa ke daftar fitur utama
feature_cols += ['NDVI', 'SWIR_ratio', 'K_proxy']
```

5. Pembersihan Data

```
# Menghapus baris yang memiliki nilai kosong (NaN) pada kolom fitur atau target.
df_clean = df.dropna(subset=[target] + feature_cols)

# Pisahkan data menjadi fitur (X) dan target (y)
X = df_clean[feature_cols]
y = df_clean[target]
```

6. Pembagian Data Latih dan Uji

```
# Data dibagi dengan rasio 80:20 (80% pelatihan, 20% pengujian)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

7. Normalisasi / Standarisasi

```
# Standarisasi diperlukan agar semua fitur memiliki skala yang sebanding.
# Ini penting karena model OLS sensitif terhadap perbedaan skala antar variabel.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

8. Tambahkan Konstanta (Intercept)

```
# Model OLS memerlukan kolom konstanta agar dapat menghitung intercept ( $\beta_0$ ).
X_train_ols = sm.add_constant(X_train_scaled)
X_test_ols = sm.add_constant(X_test_scaled)
```

9. Konversi ke DataFrame untuk interpretasi

```
# Digunakan agar kolom fitur tetap terbaca dengan jelas dalam output OLS.
cols = ['const'] + feature_cols
X_train_ols_df = pd.DataFrame(X_train_ols, columns=cols)
X_test_ols_df = pd.DataFrame(X_test_ols, columns=cols)
```

10. Pembangunan Model OLS Regression

```
# Model di-fit menggunakan data latih.
# statsmodels menghasilkan ringkasan lengkap (koefisien, p-value, R2, dll.)
model_ols = sm.OLS(y_train.reset_index(drop=True), X_train_ols_df).fit()

# Tampilkan hasil ringkasan model
print(model_ols.summary())
```

```

                        OLS Regression Results
=====
Dep. Variable:          K      R-squared:          0.432
```

```

Model:                OLS      Adj. R-squared:      0.399
Method:               Least Squares      F-statistic:      13.08
Date:                 Sun, 09 Nov 2025      Prob (F-statistic): 2.56e-40
Time:                 10:33:12      Log-Likelihood:      168.66
No. Observations:      474      AIC:      -283.3
Df Residuals:          447      BIC:      -171.0
Df Model:              26
Covariance Type:       nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
const          0.5838      0.008      72.804      0.000      0.568      0.600
Longitude      0.4128      0.211       1.961      0.050     -0.001      0.827
Latitude      -0.0765      0.147     -0.520      0.603     -0.365      0.212
b1             0.1024      0.053       1.927      0.055     -0.002      0.207
b2            -0.1529      0.122       1.251      0.212     -0.087      0.393
b3            -0.3218      0.202     -1.595      0.111     -0.718      0.075
b4             0.3128      0.130       2.397      0.017      0.056      0.569
b5            -0.2755      0.058     -4.752      0.000     -0.389     -0.162
b6            -0.2503      0.074     -3.403      0.001     -0.395     -0.106
b7             0.3186      0.060       5.352      0.000      0.202      0.436
b8            -3.0240      0.774     -3.909      0.000     -4.544     -1.504
b8a            0.6210      1.437       0.432      0.666     -2.202      3.444
b9            -0.5655      1.429     -0.396      0.693     -3.375      2.244
b11            -0.2185      0.087     -2.510      0.012     -0.390     -0.047
b12            0.0829      0.059       1.405      0.161     -0.033      0.199
Sigma_VV       -0.0159      0.070     -0.226      0.821     -0.154      0.122
Sigma_VH        0.1249      0.101       1.242      0.215     -0.073      0.323
gamma0_vv      -0.0696      0.224     -0.311      0.756     -0.509      0.370
gamma0_vh      -0.0755      0.057     -1.319      0.188     -0.188      0.037
beta0_vv       -0.0260      0.227     -0.115      0.909     -0.472      0.420
beta0_vh        0.0785      0.083       0.952      0.342     -0.084      0.241
plia          -0.7448      1.083     -0.688      0.492     -2.874      1.384
lia            0.7327      1.088       0.673      0.501     -1.406      2.872
iafe          -2.8800      0.745     -3.867      0.000     -4.344     -1.416
NDVI           0.0791      0.073       1.078      0.282     -0.065      0.223
SWIR_ratio     0.0297      0.112       0.265      0.791     -0.190      0.250
K_proxy        0.1287      0.068       1.885      0.060     -0.005      0.263
=====
Omnibus:              42.744      Durbin-Watson:      1.783
Prob(Omnibus):        0.000      Jarque-Bera (JB):      53.647
Skew:                 0.724      Prob(JB):      2.24e-12
Kurtosis:             3.786      Cond. No.      980.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

11. Prediksi Data Uji

```

# Memprediksi data uji
y_pred = model_ols.predict(X_test_ols_df)

```

12. Evaluasi Model

```

# R² (R-squared): proporsi variasi data yang dapat dijelaskan oleh model.
# RMSE: akar dari MSE, menunjukkan seberapa jauh prediksi menyimpang dari nilai aktual.
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"\nR²: {r2:.4f}, RMSE: {rmse:.4f}")

```

R²: 0.3318, RMSE: 0.1731

13. Analisis Multikolinearitas (VIF)

```

# VIF (Variance Inflation Factor) digunakan untuk mengukur korelasi antar fitur.
# Nilai VIF > 10 menunjukkan adanya multikolinearitas tinggi yang sebaiknya dihapus.
vif_data = pd.DataFrame()
vif_data["feature"] = feature_cols
vif_data["VIF"] = [variance_inflation_factor(X_train_scaled, i) for i in range(X_train_scaled.shape[1])]

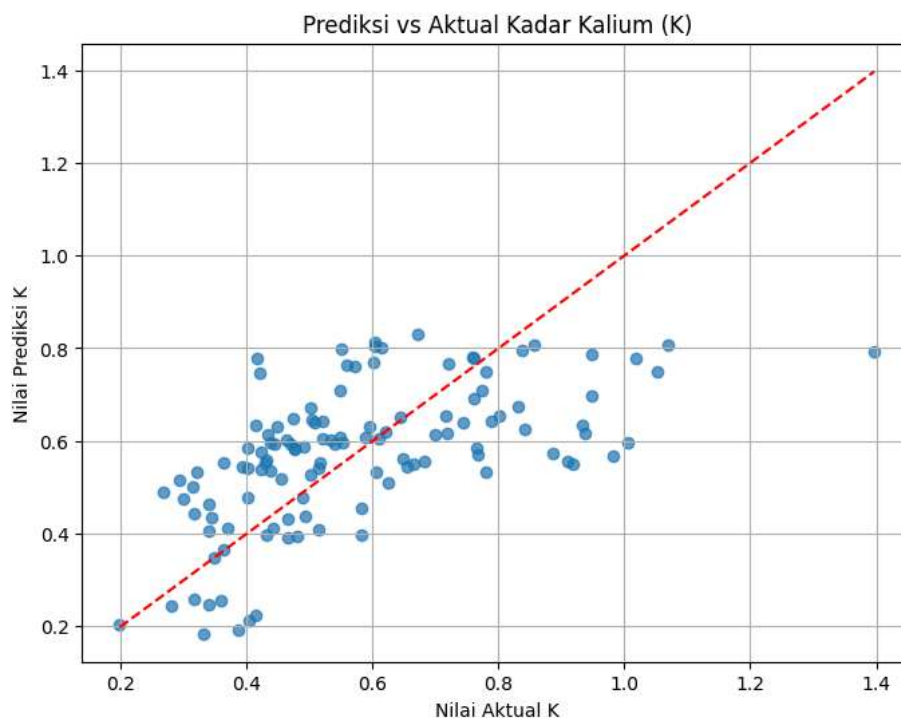
```

```
print("\nVIF (idealnya < 5):")
print(vif_data.sort_values("VIF", ascending=False))
```

```
VIF (idealnya < 5):
  feature      VIF
10    b8a 32100.872128
11    b9  31775.386318
21    lia 18426.177594
20    plia 18250.161936
9     b8  9307.004495
22    iafe 8627.284403
18  beta0_vv 801.971434
16  gamma0_vv 778.415791
0    Longitude 689.338196
4     b3  632.935849
1    Latitude 336.041067
5     b4  264.744281
3     b2  232.289753
24  SWIR_ratio 194.792953
15    Sigma_VH 157.282684
12    b11 117.858418
19  beta0_vh 105.883752
7     b6   84.125365
23    NDVI   83.776089
14    Sigma_VV 76.785210
25    K_proxy 72.517385
8     b7   55.113960
13    b12   54.164624
6     b5   52.284286
17  gamma0_vh 50.927117
2     b1   43.907918
```

14. Visualisasi Hasil Prediksi

```
# Membandingkan nilai aktual vs prediksi untuk melihat akurasi model secara visual.
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel("Nilai Aktual K")
plt.ylabel("Nilai Prediksi K")
plt.title("Prediksi vs Aktual Kadar Kalium (K)")
plt.grid(True)
plt.show()
```



15. Kesimpulan

```
# Model OLS berhasil dibangun untuk memprediksi kadar Kalium (K)
# berdasarkan kombinasi fitur citra satelit (optik, radar, dan hasil rekayasa fitur).
# Nilai R2 menunjukkan seberapa besar variasi K yang dapat dijelaskan oleh model,
# sedangkan RMSE menunjukkan tingkat error prediksi.
# Fitur dengan VIF tinggi sebaiknya dievaluasi kembali untuk menghindari multikolinearitas.
# Secara umum, model ini dapat dijadikan dasar analisis hubungan antara citra satelit
# dan kandungan unsur hara tanah, khususnya Kalium.
```